

MANUAL TÉCNICO

1. Información General

Nombre del Proyecto: Sistema de Administración de Imágenes por Capas utilizando Árboles Binarios de Búsqueda, Listas Circulares Dobles y Matrices Dispersas.

Lenguaje de Programación: Python 3

Librerías Utilizadas:

- **Pillow:** Utilizada para la manipulación y combinación de píxeles orientada a la generación física de imágenes en formato PNG.

```
from PIL import Image
from PIL import ImageDraw
```

- **Graphviz:** Utilizada para renderizar y exportar reportes gráficos de las estructuras de datos dinámicas en tiempo de ejecución.

```
from graphviz import Digraph
```

- **OS:** Módulo nativo empleado para la gestión física y depuración de los archivos de imagen generados.

```
import os
```

2. Objetivo del Sistema

El core del sistema tiene como finalidad la administración optimizada de imágenes lógicas compuestas mediante la superposición de capas. Para lograr esto de forma eficiente y en caliente, se implementan múltiples estructuras de datos dinámicas. Las operaciones principales cubren:

- Carga masiva y procesamiento de capas.
- Carga masiva de catálogos de imágenes.
- Carga masiva de registros de usuarios.
- Visualización analítica de estructuras en memoria.
- Generación automatizada de imágenes combinadas.
- Asociación relacional de imágenes a perfiles de usuarios.
- Generación integral de reportes gráficos y estados de memoria.

3. Arquitectura General del Proyecto

Organización modular y distribución de carpetas dentro del código fuente:

```
Proyecto
├── main.py
├── cargas
│   ├── carga_capas.py
│   ├── carga_imagenes.py
│   └── carga_usuarios.py
├── estructuras
│   ├── abb_capas.py
│   ├── abb_usuarios.py
│   └── lista_imagenes.py
├── modelos
│   ├── capa.py
│   ├── pixel.py
│   ├── imagen.py
│   └── usuario.py
├── reportes
│   ├── graphviz_reportes.py
│   ├── grafo_imagen.py
│   ├── grafo_usuarios.py
│   ├── grafo_lista_imagenes.py
│   └── estado_memoria.py
├── generacion
│   └── generador_imagen.py
└── archivos
    ├── capas.cap
    ├── imagenes.im
    └── usuarios.usr
```

4. Estructuras de Datos Implementadas

4.1 Árbol Binario de Búsqueda de Capas

Estructura jerárquica auto-organizada indexada por el ID único de la capa para optimizar las fases de búsqueda y renderizado.

Clase Principal: ArbolCapas

Nodo Base: NodoCapa

Operaciones Clave: Inserción ordenada, Búsqueda dicotómica de capas, Recorridos en profundidad (Inorden, Preorden, Postorden), Exportación analítica de la matriz dispersa.

4.2 Árbol Binario de Búsqueda de Usuarios

Almacena de forma alfabética los nombres únicos de usuario, permitiendo búsquedas y mutaciones rápidas en la colección.

Clase Principal: ArbolUsuarios

Nodo Base: NodoUsuario

Operaciones Clave: Inserción, Búsqueda, Eliminación de nodos con reestructuración de punteros, Asociación de imágenes exclusivas.

4.3 Lista Circular Doble de Imágenes

Catálogo secuencial y enlazado de extremos que permite una navegación bidireccional infinita sobre el listado de imágenes del sistema.

Clase Principal: ListaCircularDoble

Nodo Base: NodoImagen

Operaciones Clave: Inserción al final de la lista, Búsqueda lineal por ID, Despliegue secuencial.

4.4 Matriz Dispersas (Ortogonal)

Estructura bidimensional dinámica basada en nodos internos y listas de encabezados (Filas/Columnas) enlazadas en las 4 direcciones (Up, Down, Left, Right). Resguarda eficientemente únicamente las coordenadas que contienen un píxel con valor de color explícito, ignorando espacios vacíos.

5. Modelos del Sistema

Pixel: Unidad mínima de color. Atributos: *x*, *y*, *color*

Capa: Elemento contenedor de gráficos independientes. Atributos: *id*, *pixeles* (*Matriz Dispersa*)

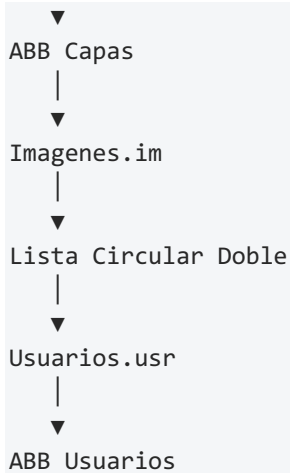
Imagen: Instancia compuesta por un conjunto ordenado de capas. Atributos: *id*, *capas* (*Colección de referencias*)

Usuario: Perfil de acceso del sistema. Atributos: *nombre*, *imagenes* (*Referencias asignadas*)

6. Flujo General del Sistema

La secuencia lógica de carga de archivos y dependencia relacional sigue el siguiente flujo estrictamente lineal:

```
Capas . cap
|
```



7. Carga Masiva de Información

Carga de Capas (capas.cap)

El parser identifica los tokens de apertura, lee las coordenadas cartesianas junto al formato hexadecimal, instancia los píxeles y los inserta ortogonalmente en la matriz dispersa antes de asociar la capa al ABB.

```
4{  
61,64, #ff0000;  
61,65, #ff0000;  
}
```

Carga de Imágenes (imagenes.im)

Mapea las colecciones estructuradas uniando capas existentes para dar de alta objetos Imagen en la lista circular doble.

```
4{4,5}  
1{6}  
2{7,8}
```

Carga de Usuarios (usuarios.usr)

Asocia identificadores de imágenes al nodo alfabético correspondiente dentro de la base de usuarios.

```
UserA:4,3;  
UserB:2;
```

8. Generación de Imágenes

El módulo generador extrae las capas de la imagen seleccionada, recorre recursivamente sus matrices de píxeles y utiliza la librería Pillow para superponer las capas en orden ascendente, aplicando el escalado y centrado correspondiente para finalmente escribir el archivo físico PNG.

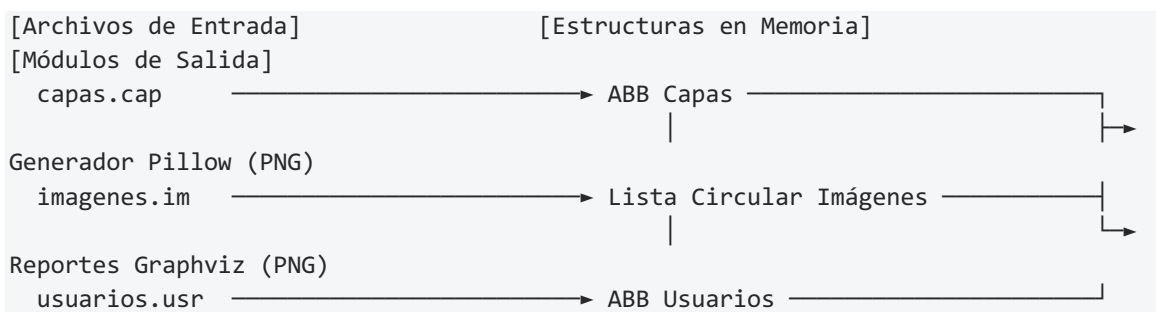
9. Reportes Generados

A través de Graphviz, el sistema genera de forma automatizada las siguientes representaciones en imágenes PNG:

- **arbol_capas.png**: Mapeo gráfico exacto de la topología del ABB de capas, mostrando nodos raíz, transiciones e hijos izquierdos y derechos.
- **lista_imagenes.png**: Grafo que representa la continuidad circular doblemente enlazada de las imágenes y la colección interna de sus capas asociadas.
- **matriz_dispersa_capa_X.png**: Visualización bidimensional completa del plano ortogonal de píxeles, incluyendo sus nodos cabecera de filas y columnas.
- **imagen_y_arbol_capas.png**: Reporte relacional que conecta los nodos de la imagen seleccionada directamente con sus nodos correspondientes dentro del ABB global.
- **arbol_usuarios.png**: Estructura jerárquica del ABB de usuarios mostrando sus perfiles y las sublistas de imágenes asignadas en memoria.
- **estado_memoria.png**: Un panel integrador que dibuja la coexistencia e interconexión de todas las estructuras lógicas simultáneamente.

10. Diagrama General del Sistema

Mapeo conceptual de componentes y dependencias de ejecución dentro del core:



11. Complejidad de Operaciones

Análisis de la complejidad algorítmica de las rutinas críticas en el peor de los casos (Notación Big O):

Operación Ejecutada

Complejidad Teórica

Insertar capa en ABB	$O(\log n)$
Buscar capa en ABB	$O(\log n)$
Insertar usuario en ABB	$O(\log n)$
Buscar usuario en ABB	$O(\log n)$
Insertar imagen en Lista Circular	$O(1)$
Recorridos del ABB (In/Pre/Post)	$O(n)$
Generar reportes estructurales	$O(n)$

12. Conclusiones

El ecosistema desarrollado logra una cohesión óptima al implementar múltiples tipos de estructuras según la naturaleza del acceso requerido: árboles binarios balanceados para búsquedas logarítmicas de entidades, listas circulares para colecciones secuenciales e infinitas, y matrices ortogonales dispersas para el ahorro crítico de memoria ram al omitir pixeles nulos. El acoplamiento con Graphviz y Pillow complementa con éxito la capa lógica convirtiendo punteros en activos visuales de alto valor técnico.